

Phenix Tutorials (Part 1 of 2)

13 documentation pages

Generated from local Phenix documentation

Contents

1. Phenix Examples and Tutorials
2. Phenix Video Tutorials
3. Tutorial: Iterative model-building, density modification and refinement starting from experimental phases
4. Tutorial: Solving a structure with cryo-EM data using docking
5. Tutorial: Solving a structure with cryo-EM data using model-building
6. Tutorial: Cryo_fit2: Fit Biomolecules into Cryo-EM Maps using dynamics (commandline)
7. Install Cryo_fit2
8. How to generate and record movie by cryo_fit2?
9. Tutorial: Fitting Biomolecules into Cryo-EM Maps using MD Simulation (command-line)
10. Tutorial: Fitting Biomolecules into Cryo-EM Maps using MD Simulation (GUI)
11. Install Cryo_fit
12. How to generate and record movie by cryo_fit1?
13. Tutorial: Generating ligand coordinates and restraints for structure refinement

Phenix Examples and Tutorials

tutorials.html

Video tutorials with data

The data for each of these tutorials are available in the GUI by selecting 'New Project' and then 'Set up Tutorial Data' and selecting a tutorial. Each video tutorial provides some background and then carries out a demonstration using the tutorial data.

Help with tutorials from the Phenix chatbot

You can use the Phenix chatbot to get help on using any Phenix program. Just ask it any question about using Phenix.

AlphaFold video tutorials

- Overview of structure determination with AlphaFold in Phenix (Introduction to the AlphaFold video tutorials)
- How to run AlphaFold prediction in Phenix (Uses sequence from Fab heavy chain tutorial)
- X-ray structure determination with AlphaFold using PredictAndBuild (Bromodomain tutorial)
- Cryo-EM structure determination with AlphaFold using PredictAndBuild (Fab heavy chain tutorial)

Crystallography video tutorials

- How to run Phaser-MR
- Using an AlphaFold model for molecular replacement
- Scaling and merging anomalous data
- Automated model building with Phenix AutoBuild
- SAD phasing using AutoSol
- Fitting ligands into density with phenix.ligandfit
- Calculating polder maps
- Simulate a SAD experiment with phenix.plan_sad_experiment
- Using Xtricator and Xtriage from HKLviewer for inspecting TNCS, anisotropy and twinning.
- HKLViewer demonstration
- Checking data quality with phenix.xtriage
- Advanced Tutorial phenix.xtriage - Space Group Identification, Ice rings & Wilson Plot
- Advanced Tutorial phenix.xtriage - translational NCS (tNCS)
- Advanced Tutorial phenix.xtriage - Checking for twinning

Validation video tutorials

- How to run MolProbity (web interface and Phenix GUI)
- MolProbity: All atom contacts tutorial

Refinement video tutorials

- How to run real-space-refine
- Phenix tutorial: phenix.refine using default values (GUI)
- Changing custom parameters in phenix.refine
- How to use secondary structure restraints
- How to use multiple refinement strategies and apply them to selected parts of a model

Model manipulation and restraints video tutorials

- Basic Phenix eLBOW tutorial
- How to run PDB tools via the GUI
- Explaining the atom selection syntax
- How to run Structure Comparison

Cryo-EM video tutorials

- Automated map sharpening with phenix.autosharpen
- Density modification for cryo-EM maps
- Automatic map interpretation with map_to_model

ChimeraX/Isolde with Phenix

- Running Phenix via ChimeraX: setup
- Running Emplace_local via ChimeraX
- Introduction to interactive low-resolution model building with ISOLDE

Sample data and instructions supplied with the GUI

Step-by-step Tutorials

- Tutorial 1: Solving a structure using SAD data
- Tutorial 2: Solving a structure using MAD data
- Tutorial 3: Solving a structure using MIR data
- Tutorial 4: Iterative model-building, density modification and refinement starting from experimental phases
- Tutorial 5: Introduction to molecular replacement, refinement, and rebuilding for novice users
- Tutorial 6: Solving a structure using Molecular Replacement
- Tutorial 7: Automatically rebuilding a structure solved by Molecular Replacement
- Tutorial 8: Fitting a flexible ligand into a difference electron density map
- Tutorial 9: Structure refinement
- Tutorial 10: Refining a structure in the presence of merohedral twinning
- Tutorial 11: Generating ligand coordinates and restraints for structure refinement

- Tutorial 12: Structure validation using MolProbity
- Tutorial 13: Running refinement examples on the command line
- Tutorial 14: Solving a cryo-EM structure by density modification, docking, refinement and loop fitting
- Tutorial 15: Solving a cryo-EM structure by density modification, model-building, refinement and loop fitting

Phenix Video Tutorials

tutorial_channel_raw.html

The Phenix video tutorials explain how to run Phenix programs and how to interpret the results. All tutorial videos are available on the Phenix tutorials YouTube channel: www.youtube.com/c/phenixtutorials. A list of all videos is also below (click on the panels to expand the view).

Overview

Map-tools

Refinement

Reflection data

Data analysis

Molecular replacement

Experimental Phasing

Other tools

Ligands

Validation

Mixed Tools

Mini-lectures

Other

Full length lectures (45 min - 1 hour)

Tutorial: Iterative model-building, density modification and refinement starting from experimental phases

autobuild.html

Introduction

This tutorial will start with experimental SAD data and density-modified phases, and carry out the process of iterative model-building, density modification and refinement with AutoBuild. It is designed to be read all the way through, giving pointers for you along the way. Once you have read it all and run the example data and looked at the output files, you will be in a good position to run your own data through AutoBuild.

Setting up to run PHENIX

If PHENIX is already installed and your environment is all set, then if you type:

```
echo $PHENIX
```

then you should get back something like this:

```
/xtal/phenix-1.3
```

If instead you get:

```
PHENIX: undefined variable
```

then you need to set up your PHENIX environment. See the PHENIX installation page for details of how to do this. If you are using the C-shell environment (csh) then all you will need to do is add one line to your .cshrc (or equivalent) file that looks like this:

```
source /xtal/phenix-1.3/phenix_env
```

(except that the path in this statement will be where your PHENIX is installed). Then the next time you log in \$PHENIX will be defined.

Running the demo p9-build data with AutoBuild

To run AutoBuild on the demo p9-build data, make yourself a tutorials directory and cd into that directory:

```
mkdir tutorials
cd tutorials
```

Now type the phenix command:

```
phenix.run_example --help
```

to list the available examples. Choosing p9-build for this tutorial, you can now use the phenix command:

```
phenix.run_example p9-build
```

to build the p9-build structure with AutoBuild. This command will copy the directory \$PHENIX/examples/p9-build to your current directory (tutorials) and call it tutorials/p9-build/. Then it will run AutoBuild using the command file run.sh that is present in this tutorials/p9-build/ directory.

This command file run.sh is simple. It says:

```
#!/bin/sh
echo "Running AutoBuild on P9 data..."
phenix.autobuild seq_file=p9.seq data=p9-solve.mtz \
```

```
input_map_file=p9-resolve.mtz resolution=2.4 \  
ncs_copies=1
```

The first line (`#!/bin/sh`) tells the system to interpret the remainder of the text in the file using the `sh` (or `bash`) -shell (`sh`).

The command `phenix.autobuild` runs the command-line version of AutoBuild (see Automated Structure Solution using AutoBuild for all the details about AutoBuild including a full list of keywords).

The arguments on the command line tell AutoBuild about the sequence file (`seq_file=p9.seq`), the data file (`data=p9-solve.mtz`), the map file with density-modified phases (`input_map_file=p9-resolve.mtz`), and the resolution `resolution=2.4` and number of ncs copies to look for (`ncs_copies=1`). (Note that each of these is specified with an `=` sign, and that there are no spaces around the `=` sign.)

Note the backslash "`\`" at the end of some of the lines in the `phenix.autobuild` command. This tells the C-shell (which interprets everything in this file) that the next line is a continuation of the current line. There must be no characters (not even a space) after the backslash for this to work.

The structure factor amplitudes and experimental phase information are in the datafile `p9-solve.mtz`. This is an `mtz` file which is a binary file that contains summary information about the dataset as well as the reflection data.

Although the `phenix.run_example p9-build` command has just run AutoBuild from a script (`run.sh`), you can run AutoBuild yourself from the command line with the same `phenix.autobuild seq_file= ...` command. You can also run AutoBuild from a GUI, or by putting commands in another type of script file. All these possibilities are described in Using the PHENIX Wizards.

Where are my files?

Once you have started AutoBuild or another Wizard, an output directory will be created in your current (working) directory. The first time you run AutoBuild in this directory, this output directory will be called `AutoBuild_run_1_` (or `AutoBuild_run_1_`, where the slash at the end just indicates that this is a directory). All of the output from run 1 of AutoBuild will be in this directory. If you run AutoBuild again, a new subdirectory called `AutoBuild_run_2_` will be created.

Inside the directory `AutoBuild_run_1_` there will be one or more temporary directories such as `TEMP0` created while the Wizard is running. The files in this temporary directory may be useful sometimes in figuring out what the Wizard is doing (or not doing!). By default these directories are emptied when the Wizard finishes (but you can keep their contents with the command `clean_up=False` if you want.)

What parameters did I use?

Once the AutoBuild wizard has started (when run from the command line), a parameters file called `autobuild.eff` will be created in your output directory (e.g., `AutoBuild_run_1_/autobuild.eff`). This parameters file has a header that says what command you used to run AutoBuild, and it contains all the starting values of all parameters for this run (including the defaults for all the parameters that you did not set).

The `autobuild.eff` file is good for more than just looking at the values of parameters, though. If you copy this file to a new one (for example `autobuild_lores.eff`) and edit it to change the values of some of the parameters (`resolution=3.0`) then you can re-run AutoBuild with the new values of your parameters like this:

```
phenix.autobuild autobuild_lores.eff
```

This command will do everything just the same as in your first run but use only the data to 3.0 Å.

Reading the log files for your AutoBuild run file

While the AutoBuild wizard is running, there are several places you can look to see what is going on. The most important one is the overall log file for the AutoBuild run. This log file is located in:

```
AutoBuild_run_1_/AutoBuild_run_1_1.log
```

for run 1 of AutoBuild. (The second 1 in this log file name will be incremented if you stop this run in the middle and restart it with a command like `phenix.autobuild run=1`).

The `AutoBuild_run_1_1.log` file is a running summary of what the AutoBuild Wizard is doing. Here are a few of the key sections of the log files produced for the p9-build SAD dataset.

Summary of the command-line arguments

Near the top of the log file you will find:

```
-----  
Starting AutoBuild with the command:
```

```
phenix.autobuild seq_file=p9.seq data=p9-solve.mtz \  
input_map_file=p9-resolve.mtz resolution=2.4 ncs_copies=1
```

This is just a repeat of how you ran AutoBuild; you can copy it and paste it into the command line to repeat this run.

Guessing the chain type

The AutoBuild Wizard will read in your sequence file and guess whether this is PROTEIN, DNA, or RNA from the sequence:

```
Guessing chain type from p9.seq  
Setting chain type to PROTEIN
```

If you want to tell the Wizard what the chain type is, you can say, `chain_type=PROTEIN`.

Guessing column labels

The AutoBuild Wizard will need to know which columns in your input data file and your input map file to use. It guesses which column labels to use and lists them out:

```
Getting column labels from p9-solve.mtz for input data file  
SG: I 4  
Cell: [113.94899749755859, 113.94899749755859, 32.4739990234375, 90.0, 90.0, 90.0]  
Input labels: ['FP', 'SIGFP', 'PHIB', 'FOM', 'HLA', 'HLB', 'HLC', 'HLD', 'None']  
  
Getting column labels from p9-resolve.mtz for input map file  
SG: I 4  
Cell: [113.94899749755859, 113.94899749755859, 32.4739990234375, 90.0, 90.0, 90.0]  
Map input labels: ['FP', 'PHIM', 'FOMM']
```

These are indeed the appropriate columns to use for experimental phases and for map coefficients, respectively. Note the "None" in the input labels for p9-solve.mtz. The last input label in this list corresponds to `FreeR_flag` and there is no Free R data in the input data file. All the data that is expected for each input file in AutoBuild can be seen in the AutoBuild web page under "Specifying which columns of data to use from input data files".

Guessing cell contents

The AutoBuild Wizard uses the sequence information in your sequence file (sequence.dat) and the cell parameters and space group to guess the number of NCS copies and the solvent fraction:

```
Number of residues in unique chains in seq file: 136
Unit cell: (113.949, 113.949, 32.474, 90, 90, 90)
Space group: I 4 (No. 79)
CELL VOLUME :421654.549593
N_EQUIV:8
GUESS OF NCS COPIES: 1
SOLVENT FRACTION ESTIMATE: 0.65
Data file (for everything including refinement): p9-solve.mtz
```

Running phenix.xtriage

The AutoBuild Wizard automatically runs phenix.xtriage on your input datafile to analyze it for twinning, outliers, translational symmetry, and other special conditions that you should be aware of. You can read more about xtriage in Data quality assessment with phenix.xtriage. The xtriage output is in the file p9-solve.mtz_xtriage.log. Part of the summary output from xtriage for this dataset looks like this:

```
The largest off-origin peak in the Patterson function is 6.40% of the
height of the origin peak. No significant pseudotranslation is detected.
```

```
The results of the L-test indicate that the intensity statistics
behave as expected. No twinning is suspected.
```

Generation of FreeR flags

The AutoBuild Wizard will create a set of free R flags indicating which reflections are not to be used in refinement. By default 5% of reflections (up to a maximum of 2000) are reserved for this test set. If you supply a reflection file with free R flags already set, then they will be used. If you want to supply a file ref.mtz specifically for refinement, you can do that with input_refinement_file=ref.mtz. Also if you want to supply a high-resolution datafile hires.mtz that has then you can do this with the keywords input_hires_file=hires.mtz. After generation of free R flags if necessary, and any merging of data files, the file to be used for refinement is called exptl_fobs_phases_freeR_flags.mtz.

Model-building with RESOLVE

The AutoBuild Wizard by default uses RESOLVE to build an atomic model of your structure.

In each cycle of model-building, the AutoBuild Wizard breaks up the building process into separate steps that can be run in parallel and then combines the results.

In the first model-building cycle, the AutoBuild Wizard builds 4 separate models by running 4 subprocesses, each of which runs the AutoBuild Wizard to just build a single model and refine it and return:

```
Build cycle 1 of 17 method:build

Build cycle 1 of 17 method:build

Running 3 parallel build jobs

Standard build in parallel
This is the first try at building this model
Setting background=False as nproc=1
Try: 1 building 1 model
Try: 2 building 1 model
Try: 3 building 1 model
Running up to 1 jobs in parallel... with total of 3 jobs
Splitting work into 3 jobs and running 1 at a time with sh in
/net/idle/scratch1/terwill/run_072908a/p9-build/AutoBuild_run_1_/TEMP0
```

```

Starting job 1...
Starting job 2...Starting job 3...
Collecting models....

Solution for try : 1 cycle_best_1.pdb
Solution 1 from build cycle 1 R= 0.24
Saving /net/idle/scratch1/terwill/run_072908a/p9-build/AutoBuild_run_1/TEMP0/
AutoBuild_run_1/cycle_best_1.pdb as MODEL_1.pdb in AutoBuild_run_1/TEMP0

Solution for try : 2 cycle_best_1.pdb
Solution 1 from build cycle 1 R= 0.24
Saving /net/idle/scratch1/terwill/run_072908a/p9-build/AutoBuild_run_1/TEMP0/
AutoBuild_run_2/cycle_best_1.pdb as MODEL_2.pdb in AutoBuild_run_1/TEMP0

Solution for try : 3 cycle_best_1.pdb
Solution 1 from build cycle 1 R= 0.24
Saving /net/idle/scratch1/terwill/run_072908a/p9-build/AutoBuild_run_1/TEMP0/
AutoBuild_run_3/cycle_best_1.pdb as MODEL_3.pdb in AutoBuild_run_1/TEMP0
Done with 3 parallel build jobs...
Running standard build to merge and extend these models now.

```

If you want to look at the log files for these individual model-building steps, you can look in the directory listed above:

```
/net/cci-filer1/vol1/tmp/terwill/phenix_examples/p9-build/AutoBuild_run_1/TEMP0
```

This will contain subdirectories with the model-building runs:

```

AutoBuild_run_1/TEMP0/AutoBuild_run_1/AutoBuild_run_1_1.log
AutoBuild_run_1/TEMP0/AutoBuild_run_2/AutoBuild_run_2_1.log
AutoBuild_run_1/TEMP0/AutoBuild_run_3/AutoBuild_run_3_1.log

```

In this case model 1 is the best. It is then used to start a merging process in which the best parts of each model are kept to create a composite model. This model is then refined, extended (by building off all the ends of chains) and saved:

```

Model completion cycle 1
Models to combine and extend: ['MODEL_1.pdb', 'MODEL_2.pdb', 'MODEL_3.pdb']
Model 1: Residues built=126 placed=118 Chains=2 Model-map CC=0.82
This is new best model with score = 240
Refining model: Build_combine_extend_1.pdb
Model: AutoBuild_run_1/TEMP0/refine_1.pdb R/Rfree=0.23/0.28

...

New best overall: AutoBuild_run_1/overall_best.pdb

```

```

Model obtained on cycle 1
R (work): 0.229221223063
R (free): 0.278562331997
Residues built: 126
Residues placed: 118
Model-map CC: 0.82
Chains: 2

```

In the second overall cycle of model building, the AutoBuild Wizard carries out several density modification steps to obtain an improved map:

```
Using coordinates of model from previous cycle in building
```

This is the key aspect of iterative model-building, density modification and refinement. The model from each cycle is used to improve the density (even in places where the model was not built) in the map for the next cycle. This is done by using density calculated from the model as part of a real-space target for statistical density modification.

The first stage of density modification uses the identification of local patterns of density unique to macromolecules (such as characteristic distances between atoms), and the presence of helices and strands in the map to improve map quality. The pattern and fragment information comes from an analysis of the map from the previous cycle and is combined into a pseudo-map combine.map. This real-space information is then merged with the experimental phase information in exptl_fobs_phases_freeR_flags.mtz:

```
Density modifying with patterns/fragments and model
Adding pattern/fragment phase information from combine.map to exptl_fobs_phases_freeR_flags.mtz
to create image.mtz
```

The composite phase information is then used in density modification. In this step the model density is used as part of the real-space target for statistical density modification. An omit map is also created that does not include the model-based information.

```
Density modifying image.mtz including model information from refine.pdb_1
to make resolve_work.mtz. Then building model
Creating omit map from image.mtz and previous models
```

A new model is then built, using the best model available so far (in this case from cycle 1), combined with pieces of a model built in four ways. The first is to "fit_loops", in which case all gaps in the model (places where the sequence file says there are residues but for which there is no model yet, and for which the residues on either side of the gap are present in the model) are systematically rebuilt. The method used is to try to build from either end of the gap and if the two chains connect with the correct number of residues then the gap is considered filled. The second method is to "connect" ends of chains. This is the same as the gap-filling procedure except that it is used in cases where the model has not been assigned to sequence so that the ends to be connected are not known in advance and the number of residues in the gap is also not known. The third method is to "build_outside", in which case the current model is used to mask out the density in the region of the model, and a model is built into the remaining density. The fourth method is simply to build a new model from scratch.

```
Standard build in parallel starting with refine.pdb_1 and ['overall_best.pdb']
```

```
Setting background=False as nproc=1
Try: 1 fit_loops=True
Try: 2 connect=True
Try: 3 build_outside=True
```

Then the current best model and the models built from each of these tries are combined together to make a composite model. As in cycle 1 it is then refined, extended, and saved (if it is an improvement):

```
Solution for try : 1 cycle_best_1.pdb
Solution 1 from build cycle 1 R= 999.9
Saving /net/idle/scratch1/terwill/run_072908a/p9-build/
AutoBuild_run_1_/TEMP0/AutoBuild_run_4_/cycle_best_1.pdb as
MODEL_1.pdb in AutoBuild_run_1_/TEMP0
```

```
Solution for try : 2 cycle_best_1.pdb
Solution 1 from build cycle 1 R= 999.9
Saving /net/idle/scratch1/terwill/run_072908a/p9-build/
AutoBuild_run_1_/TEMP0/AutoBuild_run_5_/cycle_best_1.pdb as
MODEL_2.pdb in AutoBuild_run_1_/TEMP0
```

```
Solution for try : 3 cycle_best_1.pdb
Solution 1 from build cycle 1 R= 0.23
Saving /net/idle/scratch1/terwill/run_072908a/p9-build/
AutoBuild_run_1_/TEMP0/AutoBuild_run_6_/cycle_best_1.pdb as
MODEL_3.pdb in AutoBuild_run_1_/TEMP0
Done with 3 parallel build jobs...
Running standard build to merge and extend these models now.
```

In this case the building outside-model gave a good model but the gap-filling and connecting did not fill any gaps or loops. (they gave an R of 999. meaning nothing was refined).

Model completion with these models and the current best model from cycle 1 gave:

```
Model completion cycle 1
Models to combine and extend: ['overall_best.pdb', 'starting_model.pdb',
'MODEL_1.pdb', 'MODEL_2.pdb', 'MODEL_3.pdb']
Model 1: Residues built=123 placed=107 Chains=2 Model-map CC=0.80
This is new best model with score = 226
Refining model: Build_combine_extend_1.pdb
Model: AutoBuild_run_1/TEMP0/refine_1.pdb R/Rfree=0.23/0.27
```

This process of model-building iterated with generation of real-space targets for density modification based on local patterns, fragments of structure, and the model is repeated until the R-factor does not decrease for several cycles. In this example, the best model using this procedure is obtained on cycle 2 with an R/Rfree of 0.22/0.27:

```
New best overall: AutoBuild_run_1/overall_best.pdb
Model obtained on cycle 2
R (work): 0.224655082668
R (free): 0.266763231999
Residues built: 120
Residues placed: 112
Model-map CC: 0.79
Chains: 1
```

Model-rebuilding with RESOLVE

Once the model-building procedure has converged, the AutoBuild Wizard carries out cycles of rebuilding using a slightly different protocol. The main differences in this set of cycles are that the local patterns and fragments approaches are no longer used (the maps by this time look so much like a macromolecule that these procedures do not add anything), and that the starting point for density modification is a model-based map, not the experimental map.

In this example, the rebuilding steps improve the model just a little, and the process ends after 2 cycles of rebuilding:

```
New best overall: AutoBuild_run_1/overall_best.pdb
Model obtained on cycle 2
R (work): 0.224655082668
R (free): 0.266763231999
Residues built: 120
Residues placed: 112
Model-map CC: 0.79
Chains: 1
```

The AutoBuild_summary.dat summary file

A quick summary of the results of your AutoBuild run is in the AutoBuild_summary.dat file in your output directory. This file lists the key files that were produced in your run of AutoBuild (all these are in the output directory) and some of the key statistics for the run. Here is the summary for this p9-build model-building run:

```
Summary of model-building for run 1 Sun Jun 29 12:02:56 2008
Files are in the directory: /net/idle/scratch1/terwill/run_072908a/p9-build/AutoBuild_run_1/

Starting mtz file: /net/idle/scratch1/terwill/run_072908a/p9-build/p9-solve.mtz
Sequence file: /net/idle/scratch1/terwill/run_072908a/p9-build/p9.seq

Best solution on cycle: 4 R/Rfree=0.22/0.27

Summary of output files for Solution 3 from rebuild cycle 4
```

```

--- Model (PDB file) ---
pdb_file: AutoBuild_run_1_/cycle_best_4.pdb

--- Refinement log file ---
log_refine: AutoBuild_run_1_/cycle_best_4.log_refine

--- Model-building log file ---
log: AutoBuild_run_1_/cycle_best_4.log

--- Model-map correlation log file ---
log_eval: AutoBuild_run_1_/cycle_best_4.log_eval

--- 2FoFc and FoFc map coefficients from refinement 2FOFCWT PH2FOFCWT FOFCWT PHFOFCWT ---
refine_map_coeffs: AutoBuild_run_1_/cycle_best_refine_map_coeffs_4.mtz

--- Data for refinement FP SIGFP PHIM FOMM HLAM HLBM HLCM HLDM FreeR_flag ---
hklout_ref: AutoBuild_run_1_/exptl_fobs_phases_freeR_flags.mtz

--- Density-modification log file ---
log_denmod: AutoBuild_run_1_/cycle_best_4.log_denmod

--- Density-modified map coefficients FP PHIM FOM ---
hklout_denmod: AutoBuild_run_1_/cycle_best_4.mtz

```

You might consider making one very good model now with:

```

phenix.autobuild \
  data=AutoBuild_run_1_/exptl_fobs_phases_freeR_flags.mtz \
  model=AutoBuild_run_1_/cycle_best_4.pdb \
  rebuild_in_place=True \
  seq_file=/net/idle/scratch1/terwill/run_072908a/p9-build/p9.seq

```

```

SOLUTION CYCLE R RFREE BUILT PLACED
1 1 0.23 0.28 126 118
2 2 0.22 0.27 120 112
3 4 0.22 0.27 123 111
4 5 0.22 0.26 121 113

```

Note that the file AutoBuild_run_1_/cycle_best_4.log_eval in this example has a complete analysis of the the fit of the model in AutoBuild_run_1_/cycle_best_4.pdb to the best map in AutoBuild_run_1_/cycle_best_4.log_denmod. This is useful in identifying places where additional rebuilding needs to be done.

Creating an improved model after AutoBuild

In our example, the summary file had this phrase in it:

You might consider making one very good model now with:

```

phenix.autobuild \
  data=AutoBuild_run_1_/exptl_fobs_phases_freeR_flags.mtz \
  model=AutoBuild_run_1_/cycle_best_4.pdb \
  rebuild_in_place=True \
  seq_file=/net/cci-filer1/vol1/tmp/terwill/phenix_examples/p9-build/p9.seq

```

This is a command-line command to take the final model from this AutoBuild run, rebuild it 4 times using the "rebuild_in_place" algorithm, combine the resulting models to make a composite model, refine it, and write out the final model. This method is very effective at improving models from AutoBuild (or from any other source). You can see more details of this in Tutorial 6: Automatically rebuilding a structure solved by Molecular Replacement

How do I know if iterative model-building, density modification and refinement worked?

Here are some of the things to look for to tell if you have obtained a good model:

- How much of the model was built? For the AutoBuild Wizard, you should expect more than 90% of your model to be built in favorable cases when the resolution of the data is 3 Å or better. Have a look at the model. It should have all the appropriate features of a protein or nucleic acid.
- What is the R-factor and the free R-factor of the model? For a solution at moderate to high resolution (2.5 Å or better) the R-factor should be in the low 20's and the free R-factor about 0.05 higher to be very good. For lower-resolution data, an R-factor in the low 30's is probably largely correct but the model is not very good.
- Does the model show NCS if there are more than one copies in the asymmetric unit? You can look at your log file which will tell you about any NCS in your model.

What to do next

Once you have run AutoBuild and have obtained a good model, you will want to inspect and touch up the model carefully, rebuilding any parts of the model that do not agree well with the final map. You should also have a close look at all the solvent molecules in your model, making sure that they all have reasonable relationships to the macromolecule and to each other, and that they are not simply filling up density where a ligand or the macromolecule really goes.

The next thing to do is to add in any ligands (metals, cofactors) if there is density for them. You can use the LigandFit Wizard (see Automated Ligand Fitting using LigandFit) to help you fit ligands into your map automatically.

If you do not obtain a good model, then it's not quite time to give up yet. There are a number of standard things to try that may improve the model building. Here are a few that you should try:

- Have a careful look at all the output files. Work your way through the main log file (e.g., AutoBuild_run_1_1.log). Is there anything strange or unusual in any of them that may give you a clue as to what to try next? What does the xtriage output say? Is there twinning or strong translational symmetry? Are there problems with reflections near ice rings? Are there many outlier reflections?
- Have a look at the electron density maps at different stages in model building. How does the starting map look? How about the maps produced during the iterative building process (cycle_best_xx.mtz)? If the maps look poor, then the rebuilding process will not work well. However if the maps look good and the rebuilding is not working well then it is worth trying to identify where the problem is.
- Try a different resolution cutoff. You may be able to use higher-resolution data in AutoBuild than you used in structure solution.

Additional information

For details about the AutoBuild Wizard, see Automated Model building and Rebuilding using AutoBuild. For help on running Wizards, see Using the PHENIX Wizards.

Tutorial: Solving a structure with cryo-EM data using docking

[cryo_em_structure_solution_docking.html](#)

This example is density modification of a cryo-EM map, followed by optional structure determination by docking a homologous structure

The data are half-maps for apoferritin (EMD entry 20026, 1.8 Å)

The files are:

emd_20026_half_map_1_box.ccp4 emd_20026_half_map_2_box.ccp4 -- half maps
seq.dat -- sequence file
20026_auto_sharpen_A.ccp4 -- deposited map, half-map sharpened
apoferritin_chainA_rsr.pdb -- model refined against sharpened map
1aew_A.pdb -- chain A from X-ray structure of horse apoferritin (53% identical to human apoferritin)
run_denmod_dock.csh -- command file that will run all these things for you if you want

- emd_20026_half_map_1_box.ccp4 emd_20026_half_map_2_box.ccp4 -- half maps
- seq.dat -- sequence file
- 20026_auto_sharpen_A.ccp4 -- deposited map, half-map sharpened
- apoferritin_chainA_rsr.pdb -- model refined against sharpened map
- 1aew_A.pdb -- chain A from X-ray structure of horse apoferritin (53% identical to human apoferritin)
- run_denmod_dock.csh -- command file that will run all these things for you if you want

The maps are all cut out (boxed) from the deposited maps (just to make the files smaller for this demo).

- DENSITY MODIFICATION OF HALF-MAPS

To run one cycle of density modification from the command line you can paste this in to a terminal window. From the GUI just set all the same parameters and no need to set the output_directory (it will go in resolve_cryo_em_0 or a similar directory name):

```
phenix.resolve_cryo_em emd_20026_half_map_1_box.ccp4
```

```
emd_20026_half_map_2_box.ccp4 resolution=2.2 dm_resolution=1.8 seq_file=seq.dat cycles=1 nproc=4  
box_before_analysis=False output_directory=denmod  
density_modify_unsharpened_maps=True initial_map_file_name=initial_map.ccp4
```

In about 15 minutes this will produce (in a resolve_cryo_em_xx directory) the file:

```
denmod/denmod_map.ccp4 -- density-modified map
```

You may want to cut out the parts of your map that match apoferritin_chainA_rsr.pdb:

```
phenix.map_box ignore_symmetry_conflicts=True
```

```
apoferritin_chainA_rsr.pdb denmod/denmod_map.ccp4 prefix=denmod_map_A
```

We are saying "ignore_symmetry_conflicts=True" because the map in denmod/denmod_map.ccp4 may have a different "unit cell" than apoferritin_chainA_rsr.pdb. Here the "unit cell" is just a description of the box of points that are in the map. It can sometimes refer to the size of the original map and sometimes to the size of the working part of the map and this command says to ignore these differences.

In Coot or Chimera you can load in the model (apoferritin_chainA_rsr.pdb) and compare the sharpened deposited map (20026_auto_sharpen_A.ccp4) with the density modified map (denmod_map_A.ccp4). Look at residues 13 and 81 and the main chain near residue 161 for examples of substantial improvement in the map.

- STRUCTURE SOLUTION BY DOCKING HOMOLOGOUS MODEL (horse apoferritin, 53% identical)

The file 1aew_A.pdb contains the X-ray structure of 1aew, a homologous protein from horse (53% identical, rmsd 0.5 Å). We are going to dock this structure into our density-modified map, refine it, fix the sequence, refine again, and density modify using the model to get a final map.

1. Getting map symmetry We will want a symmetry file describing the symmetry operators that have been used in our map. We can get them automatically in about 30 seconds with:

```
phenix.map_symmetry denmod/denmod_map.ccp4 resolution=2
```

```
symmetry_out=denmod_map.ncs_spec
```

Now denmod_map.ncs_spec has the 24 symmetry operators for octahedral symmetry in the appropriate location of this map.

- Docking into density-modified map

We can find where 1aew_A.pdb fits in our map with dock_in_map:

```
phenix.dock_in_map denmod/denmod_map.ccp4 1aew_A.pdb resolution=2 nproc=4
```

```
symmetry_file=denmod_map.ncs_spec pdb_out=1aew_A_docked_in_denmod_map.pdb
```

In a minute or two you should get the model 1aew_A_docked_in_denmod_map.pdb.

- Cut out a piece of density-modified map and compare to docked model

You can cut out a piece of your density modified map and look at it along with this model this way:

```
phenix.map_box denmod/denmod_map.ccp4 1aew_A_docked_in_denmod_map.pdb  
ignore_symmetry_conflicts=true
```

This will produce 1aew_A_docked_in_denmod_map_box.ccp4 and

1aew_A_docked_in_denmod_map_box.pdb which you can look at in Coot or Chimera. Notice that a lot of the side chains do not match the density because the model is from a homologous structure.

- Refine the model against the density-modified map

We can refine the docked model either against the original or cut-out map. Let's use the cut-out map. We can also refine just chain A or the whole 24-mer. The difference is if you use the whole 24-mer you are going to include inter-chain contacts and if you use a single chain you will ignore them. Let's use just one chain for now:

```
phenix.real_space_refine 1aew_A_docked_in_denmod_map.pdb  
1aew_A_docked_in_denmod_map_box.ccp4 resolution=2 ignore_symmetry_conflicts=True
```

After a couple minutes we have the output model:

1aew_A_docked_in_denmod_map_real_space_refined.pdb. If you look at this model in Coot or Chimera you'll see it has moved some side chains into density, but that they still don't match the density because the sequence of this docked model is not the sequence of human apoferritin (look at R29 for example which should be a tyrosine).

- Fix the sequence

We can redo the sequence of our docked model with sequence_from_map:

```
phenix.sequence_from_map 1aew_A_docked_in_denmod_map_real_space_refined.pdb
```

```
1aew_A_docked_in_denmod_map_box.ccp4 resolution=2 seq_unique.dat nproc=4  
pdb_out=1aew_A_docked_rsr_seq.pdb
```



```
1aew_A_docked_in_denmod_map_box.ccp4 resolution=2 seq_unique.dat nproc=4  
pdb_out=1aew_A_docked_rsr_seq.pdb
```

In a couple minutes you should get the file: 1aew_A_docked_rsr_seq.pdb with the sequence of human apoferritin mapped onto the real-space-refined model. Have a look at this with the map now in Coot or Chimera. You can see that some of the side chains need refinement and that a loop by P157 needs to be moved. Let's first just refine our new model.

- Refine model with new sequence

We can refine just the same as before but with our new sequence:

```
phenix.real_space_refine 1aew_A_docked_rsr_seq.pdb 1aew_A_docked_in_denmod_map_box.ccp4  
resolution=2 ignore_symmetry_conflicts=True
```

In a couple minutes we have the refined model: 1aew_A_docked_rsr_seq_real_space_refined.pdb. This model fits the map pretty well, but the loop at P157 is still way off. We need to re-fit this loop.

- Fit a loop that is not in density

We can fix this loop using the fit_loops tool. We can look at the model and map and notice that the part of the model that doesn't fit is residues 155-159. Let's fix these with:

```
phenix.fit_loops 1aew_A_docked_rsr_seq_real_space_refined.pdb  
map_in=1aew_A_docked_in_denmod_map_box.ccp4 resolution=2 seq_unique.dat remove_loops=True  
start=155 end=159 pdb_out=1aew_A_docked_rsr_seq_rsr_loop157.pdb
```

In a minute this should fit this part of the chain into the density.

- Refine model with fitted loop

We can refine one more time after fitting our loop at P157:

```
phenix.real_space_refine 1aew_A_docked_rsr_seq_rsr_loop157.pdb  
1aew_A_docked_in_denmod_map_box.ccp4 resolution=2 ignore_symmetry_conflicts=True
```

Now we have 1aew_A_docked_rsr_seq_rsr_loop157_real_space_refined.pdb. Have a look at it in Coot or Chimera.

Tutorial: Solving a structure with cryo-EM data using model-building

[cryo_em_structure_solution_model_building.html](#)

This example is density modification of a cryo-EM map, followed by structure determination by automatic model-building, fixing a mis-traced segment, sequence assignment and refinement

NOTE: the details of what needs to be fixed after automatic model-building (steps 5-11) may change with platform, number of processors or version of Phenix used

The data are half-maps for apoferritin (EMD entry 20026, 1.8 Å)

The files are (NOTE: the map files are in the apoferritin_denmod directory):

emd_20026_half_map_1_box.ccp4 emd_20026_half_map_2_box.ccp4 -- half maps
seq.dat -- sequence file
run_denmod_model_building.csh -- script to run all these commands

- emd_20026_half_map_1_box.ccp4 emd_20026_half_map_2_box.ccp4 -- half maps
- seq.dat -- sequence file
- run_denmod_model_building.csh -- script to run all these commands

The maps are all cut out (boxed) from the deposited maps (just to make the files smaller for this demo).

- DENSITY MODIFICATION OF HALF-MAPS

To run one cycle of density modification from the command line you can paste this in to a terminal window. From the GUI just set all the same parameters and no need to set the output_directory (it will go in resolve_cryo_em_0 or a similar directory name):

```
phenix.resolve_cryo_em emd_20026_half_map_1_box.ccp4
```

```
emd_20026_half_map_2_box.ccp4 resolution=2.2 dm_resolution=1.8 seq_file=seq.dat cycles=1 nproc=4  
box_before_analysis=False density_modify_unsharpened_maps=True output_directory=denmod  
output_files.denmod_map_file_name=denmod_map.ccp4 initial_map_file_name=initial_map.ccp4
```

In about 15 minutes this will produce (in a denmod/ or

resolve_cryo_em_xx directory) the file: denmod_map.ccp4

- STRUCTURE SOLUTION BY AUTOMATED MODEL-BUILDING

We are going to build a model into our density-modified map,

refine it, fix the sequence and refine again.

1. Getting map symmetry We will want a symmetry file describing the symmetry operators that have been used in our map. We can get them automatically in about 30 seconds with:

```
phenix.map_symmetry denmod/denmod_map.ccp4 resolution=2  
symmetry_out=denmod_map.ncs_spec
```

Now denmod_map.ncs_spec has the 24 symmetry operators for octahedral symmetry in the appropriate location of this map.

- Automated model-building using a density-modified map

We can build a model into our map with map_to_model:

```
phenix.map_to_model denmod/denmod_map.ccp4 resolution=2 nproc=4
```

```
ncs_file=denmod_map.ncs_spec pdb_out=map_to_model.pdb seq_file=seq.dat
```

In a half hour or so you should get the model 1aew_A_built_in_denmod_map.pdb.

3. Cut out a piece of density-modified map and compare to build_in model You can cut out a piece of your density modified map and look at it along with your model this way:

```
phenix.map_box denmod/denmod_map.ccp4 map_to_model.pdb
```

```
ignore_symmetry_conflicts=true prefix=denmod_map_A
```

This will produce denmod_map_A.ccp4 and denmod_map_A.pdb which you can look at in Coot or Chimera. The model is mostly pretty good but may need some fixing, depending on your version of Phenix

- Refine the model against the density-modified map

We can refine the build_in model either against the original or cut-out map.

Let's use the cut-out map. We can also refine just chain A or the whole 24-mer. The difference is if you use the whole 24-mer you are

going to include inter-chain contacts and if you use a single chain you

will ignore them. Let's use just one chain for now:

```
phenix.real_space_refine map_to_model.pdb denmod_map_A.ccp4 resolution=2
```

```
ignore_symmetry_conflicts=True
```

After a couple minutes we have the output model: map_to_model_real_space_refined.pdb. If you look at this model in Coot or Chimera you'll see it has moved some side chains into density, but that the model still doesn't match the density for residues 154-162.

5. If your model has parts that do not fit, try something like the following.

NOTE: the results for these steps may depend on platform, version, and number of processors so you may have to adjust them based on what your model looks like.

First remove the bad part (here the bad part is residues 154 through 162):

```
phenix.pdbtools map_to_model_real_space_refined.pdb
```

```
remove="resseq 154:162" output.file_name=map_to_model_real_space_refined_del.pdb
```

6. Redo the sequence assignment and see if we can fill in the gap. Here the keyword run_assign_sequence=True is used to reassign the sequence and then fill in the gap:

```
phenix.sequence_from_map run_assign_sequence=True
```

```
model_file=map_to_model_real_space_refined_del.pdb seq_file=seq_unique.dat
```

```
map_file=denmod_map_A.ccp4 pdb_out=sequence_from_map.pdb resolution=2
```

This should produce the file sequence_from_map.pdb in a couple minutes.

- We can fix one other part of the model that does not fit well (residues 6-9):

```
phenix.pdbtools sequence_from_map.pdb
```

```
remove="resseq 6:9" output.file_name=sequence_from_map_del_6-9.pdb
```

- Redo the sequence assignment and see if we can fill in the gap as in 6 above:

```
phenix.sequence_from_map run_assign_sequence=True
```

```
model_file=sequence_from_map_del_6-9.pdb seq_file=seq_unique.dat map_file=denmod_map_A.ccp4  
pdb_out=sequence_from_map_2.pdb resolution=2
```

This should produce the file sequence_from_map_2.pdb in a couple minutes.

- Let's refine one more time:

```
phenix.real_space_refine sequence_from_map_2.pdb denmod_map_A.ccp4  
resolution=2 ignore_symmetry_conflicts=True
```

Now have a look at sequence_from_map_2_real_space_refined.pdb and denmod_map_A.ccp4 in Coot or Chimera. The model should fit the map quite well. If there are missing side chains, you can fill them in with:

- Run sequence_from_map

```
phenix.sequence_from_map sequence_from_map_2_real_space_refined.pdb  
denmod_map_A.ccp4 seq_file=seq_unique.dat resolution=2 pdb_out=sequence_from_map_3.pdb
```

and now sequence_from_map_3.pdb should have more side chains built. You could refine once more too:

- Final refinement:

```
phenix.real_space_refine sequence_from_map_3.pdb denmod_map_A.ccp4  
resolution=2 ignore_symmetry_conflicts=True
```

and sequence_from_map_3_real_space_refined.pdb is the refined model. Next you would go over the model carefully using validation tools and remove any parts that are not ok and fix parts that are not quite right. You can also create the full 24-mer using your map symmetry and the model

Tutorial: Cryo_fit2: Fit Biomolecules into Cryo-EM Maps using dynamics (commandline)

[cryo_fit2_cmdline.html](#)

Contents

- Note
- Overview
- Theory for cryo_fit2
- Input
- How to run
- How to add user specified geometries
- Output as a static structure
- How to show map contour as mesh
- How to generate and record movie?
- FAQ
- Options
- List of most useful options

Note

cryo_fit2 is under active development.

Therefore, consider to use cryo_fit1 instead (especially for RNA modeling).

Overview

This tutorial will show you how to fit biomolecule atomic structures into cryo-EM maps using dynamics simulation with PHENIX commandline

Theory for cryo_fit2

See the theory notes for cryo_fit2

Input

- <initial_model>, <target_map> and <map resolution>

The <initial_model> is a guide or template structure (.cif/mmCIF/.pdb) that is close to a <target_map> (.ccp4/.map) structurally. A user can use map_to_model to build the atomic model. A user can use either dock_in_map or UCSF chimera (Tools->Volume Data->Fit in Map) to roughly place the model into a map. The <target_map> should have decent density. Most mrc type cryo-EM maps have decent density. However, some maps (like from UCSF Chimera's map segmentation) have low density. Therefore, cryo_fit2 will have just little incentive to fit to it. For example, left yellow maps are fine. However, right gray/white maps lack enough density to run. To fit into these low density maps, Doo Nam tried very high map_weights (like 10^8). However, these efforts either broke atomistic model geometries or did not budge the atomistic model.

- The <initial_model> is a guide or template structure (.cif/mmCIF/.pdb) that is close to a <target_map> (.ccp4/.map) structurally.

A user can use map_to_model to build the atomic model. A user can use either dock_in_map or UCSF chimera (Tools->Volume Data->Fit in Map) to roughly place the model into a map.

- A user can use map_to_model to build the atomic model.
- A user can use either dock_in_map or UCSF chimera (Tools->Volume Data->Fit in Map) to roughly place the model into a map.
- The <target_map> should have decent density.

Most mrc type cryo-EM maps have decent density. However, some maps (like from UCSF Chimera's map segmentation) have low density. Therefore, cryo_fit2 will have just little incentive to fit to it. For example, left yellow maps are fine. However, right gray/white maps lack enough density to run. To fit into these low density maps, Doo Nam tried very high map_weights (like 10^8). However, these efforts either broke atomistic model geometries or did not budge the atomistic model.

- Most mrc type cryo-EM maps have decent density.
- However, some maps (like from UCSF Chimera's map segmentation) have low density. Therefore, cryo_fit2 will have just little incentive to fit to it.

For example, left yellow maps are fine. However, right gray/white maps lack enough density to run.

- For example, left yellow maps are fine. However, right gray/white maps lack enough density to run.

For example, left yellow maps are fine. However, right gray/white maps lack enough density to run.

- To fit into these low density maps, Doo Nam tried very high map_weights (like 10^8). However, these efforts either broke atomistic model geometries or did not budge the atomistic model.

How to run

At command line:

```
% phenix.cryo_fit2 <initial_model> <target_map> <map resolution in Angstrom>
```

For example:

```
% phenix.cryo_fit2 model.pdb map.ccp4 3.5
```

Tutorial command:

```
% Go to <user phenix>/modules/cryo_fit2/tutorial
```

```
% source run_me.sh
```

```
%% (run_me.sh is phenix.cryo_fit2 input/tutorial_cryo_fit2_model.pdb input/tutorial_cryo_fit2_map.ccp4 resolution)
```

How to add user specified geometries

- There are two methods of generating a user specified geometry file (.eff file).

If a user specified geometry file is small, then simply copy/modify <user phenix>/modules/cryo_fit2/tutorial/eff_file_template/user_custom_geom.eff, and modify it for a user's need. If a user specified geometry file is large, then run: % phenix.cryo_fit2 <initial_model> <target_map> <map resolution in Angstrom> <write_custom_geom_only=True> <HE_angle_sigma_scale> <HE_sigma> <parallelity_sigma> For example: % phenix.cryo_fit2 model.pdb map.ccp4 3.5 write_custom_geom_only=True HE_angle_sigma_scale=0.04 HE_sigma=0.2 parallelity_sigma=0.02 Then,

automatically generated geometry file should be generated at the same folder. Modify it for a user's need. Other than write_custom_geom_only, all parameters are optional (if a user didn't specify these, cryo_fit2 will use default values).

- If a user specified geometry file is small, then simply copy/modify <user phenix>/modules/cryo_fit2/tutorial/eff_file_template/user_custom_geom.eff, and modify it for a user's need.

If a user specified geometry file is small, then simply copy/modify <user phenix>/modules/cryo_fit2/tutorial/eff_file_template/user_custom_geom.eff, and modify it for a user's need.

- If a user specified geometry file is large, then run: % phenix.cryo_fit2 <initial_model> <target_map> <map resolution in Angstrom> <write_custom_geom_only=True> <HE_angle_sigma_scale> <HE_sigma> <parallelity_sigma>

If a user specified geometry file is large, then run:

```
% phenix.cryo_fit2 &lt;initial_model> &lt;target_map> &lt;map resolution in Angstrom> &lt;write_custom_geom_only=True>
```

For example: % phenix.cryo_fit2 model.pdb map.ccp4 3.5 write_custom_geom_only=True HE_angle_sigma_scale=0.04 HE_sigma=0.2 parallelity_sigma=0.02 Then, automatically generated geometry file should be generated at the same folder. Modify it for a user's need. Other than write_custom_geom_only, all parameters are optional (if a user didn't specify these, cryo_fit2 will use default values).

For example:

```
% phenix.cryo_fit2 model.pdb map.ccp4 3.5 write_custom_geom_only=True HE_angle_sigma_scale=0.04 HE_sigma=0.2 parallelity_sigma=0.02
```

Then, automatically generated geometry file should be generated at the same folder.

Modify it for a user's need.

Other than write_custom_geom_only, all parameters are optional (if a user didn't specify these, cryo_fit2 will use default values).

- Once a user generated an eff file (by any method), then provide it to cryo_fit2 like: % phenix.cryo_fit2 <initial_model> <target_map> <map resolution in Angstrom> <eff file>

Once a user generated an eff file (by any method), then provide it to cryo_fit2 like:

```
% phenix.cryo_fit2 &lt;initial_model> &lt;target_map> &lt;map resolution in Angstrom> &lt;eff file>
```

For example: % phenix.cryo_fit2 model.pdb map.ccp4 3.5 user_custom_geom.eff

- For example: % phenix.cryo_fit2 model.pdb map.ccp4 3.5 user_custom_geom.eff

For example:

```
% phenix.cryo_fit2 model.pdb map.ccp4 3.5 user_custom_geom.eff
```

Output as a static structure

A final cryo_fitted structure: output/cryo_fit2_fitted.pdb

How to show map contour as mesh

In pymol commandline:

```
% load ./user_map.ccp4, map1, 1 , ccp4
% isomesh mesh1, map1, 1.5, (all), 0, 1, 3.0
```

```
% cmd.color("blue", "mesh1")
```

Otherwise, use pymol GUI:

```
% A (action) -&gt; Mesh -&gt; @ Level 3.0
```

How to generate and record movie?

Generate_record_movie_by_cryo_fit2

FAQ

See the cryo_fit2 FAQ

Options

All options will be used as default if unspecified. e.g. secondary_structure_enabled = True

List of most useful options

Install Cryo_fit2

cryo_fit2_install.html

Contents

- Note
- 1. Install Phenix
- 2. Link cryo_fit2 to phenix
- 3. Troubleshooting
- 4. Test run

Note

cryo_fit2 is under active development.

Please consider to use cryo_fit1 instead.

1. Install Phenix

See the installation notes for Phenix

2. Link cryo_fit2 to phenix

At <user phenix>/modules

```
git clone git@github.com:cryoFIT/cryo_fit2.git
```

At any folder,

```
libtbx.configure cryo_fit2
```

3. Troubleshooting

- If user's OS doesn't have git install git
- install git
- If a user sees the error message during git clone, "Permission denied (publickey).fatal: Could not read from remote repository. Please make sure you have the correct access rights and the repository exists.", user's computer needs to be registered to his/her github account. Therefore, make github account (free), Generate a new SSH key and Add a new SSH key to your GitHub account
- user's computer needs to be registered to his/her github account. Therefore, make github account (free), Generate a new SSH key and Add a new SSH key to your GitHub account
- If user's macOS shows, "xcode-select: note: no developer tools were found at '/Applications/Xcode.app', requesting install. Choose an option in the dialog to download the command line developer tools." download commandline as instructed (this may take ~7 minutes)
- download commandline as instructed (this may take ~7 minutes)
- "-bash: libtbx.configure: command not found" (in commandline) export
PATH="/Users/.../phenix-xxxx/build/bin":\$PATH source /Users/.../phenix-xxxx/phenix_env.sh
libtbx.configure cryo_fit2

- (in commandline)
- export PATH="/Users/.../phenix-xxxx/build/bin":\$PATH
- source /Users/.../phenix-xxxx/phenix_env.sh
- libtbx.configure cryo_fit2

4. Test run

See the cryo_fit2 tutorial

How to generate and record movie by cryo_fit2?

cryo_fit2_movie.html

Contents

- Output as a movie
- How to save a movie file (confirmed with MacPymol 2.2.2)
- Note

Output as a movie

- When an record_states option is True, structures of intermediate steps that can be animated with pymol play button are save as output/all_states.pdb
- Open this pdb file with pymol, then click play button (triangle at right bottom) plays the movie (confirmed with MacPymol 2.2.2).

(note. A below difference is observed with MacPymol ver. 2.2.2) If a user dragged all_states.pdb into pymol after opening it, the biomolecule structure looks simplified and movie playing is fast. If a user dragged all_states.pdb into pymol icon, pymol opens along with all_states.pdb, the biomolecule structure is more detailed, and movie playing is slow.

- (note. A below difference is observed with MacPymol ver. 2.2.2)

If a user dragged all_states.pdb into pymol after opening it, the biomolecule structure looks simplified and movie playing is fast. If a user dragged all_states.pdb into pymol icon, pymol opens along with all_states.pdb, the biomolecule structure is more detailed, and movie playing is slow.

- If a user dragged all_states.pdb into pymol after opening it, the biomolecule structure looks simplified and movie playing is fast.
- If a user dragged all_states.pdb into pymol icon, pymol opens along with all_states.pdb, the biomolecule structure is more detailed, and movie playing is slow.

How to save a movie file (confirmed with MacPymol 2.2.2)

As a user is playing a movie (by clicking a bottom right triangle movie playing button),

File -> Export Movie As -> MPEG

Note

- If a number MD steps is too many, memory requirement is too much.

For example, 3k atoms molecules (like L1 stalk in a ribosome) required more than 160 GB of memory.

- For example, 3k atoms molecules (like L1 stalk in a ribosome) required more than 160 GB of memory.

Tutorial: Fitting Biomolecules into Cryo-EM Maps using MD Simulation (command-line)

[cryo_fit_cmdline.html](#)

Contents

- Overview
- Input files
- Basic command-line syntax
- Running the tutorial with provided structure and map
- Output
- Example
- Limitation
- How to generate and record movie?
- Options
- List of the most useful options
- List of all available keywords

Overview

This tutorial will show you how to fit biomolecular structures into cryo-EM maps using molecular dynamics simulation with Cryo_fit under the PHENIX command-line environment.

For GUI tutorial, please see the [cryo_fit_gui_tutorial](#).

Theoretical explanation of Cryo_fit is [here](#).

For Cryo_fit installation, please see installation notes for [cryo_fit](#).

Input files

<initial_model> and <target_map>

Initial Model: .cif or .pdb

The initial model is a guide or template structure (CIF/mmCIF/pdb) that is close to the model representing the target cryo EM map structurally.

Target Map: .ccp4, .map/.mrc (MRC style binary file), or .sit (Sitius style text file)

Basic command-line syntax

- `phenix.cryo_fit <initial_model> <target_map>`

For example, `phenix.cryo_fit model.pdb map.ccp4`

- For example, `phenix.cryo_fit model.pdb map.ccp4`

Running the tutorial with provided structure and map

- Use the map and model files in \$PHENIX/modules/cryo_fit/tutorial_input_files

Command:

```
phenix.cryo_fit GTPase_activation_center_tutorial.pdb GTPase_activation_center_tutorial_gaussian_1p5.mrc
```

Output

The "Output" folder has all the useful output files

Cryo_fit saves 3 structures with the highest correlation coefficients (CC):

cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb extracted_<x>_ps.pdb
extracted_<y>_ps.pdb

- cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb
- extracted_<x>_ps.pdb
- extracted_<y>_ps.pdb

These are

The final cryo_fitted structure (The PDB file with the highest CC):

cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb A <x>.gro file is for vmd visualization. A geometry extracted PDB file at certain (x) pico second during MD simulation:
extracted_<x>_ps.pdb

- The final cryo_fitted structure (The PDB file with the highest CC):

cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb

- cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb
- A <x>.gro file is for vmd visualization.
- A geometry extracted PDB file at certain (x) pico second during MD simulation:

extracted_<x>_ps.pdb

- extracted_<x>_ps.pdb

If Cryo_fit cannot find a structure with higher CC than that of the starting model, the input PDB file will exist in the output folder.

Also, the history of the CCs between cryo_fitted structures and the cryo-EM map is stored in the "cc_record" file.

- In summary, the PDB file with the highest structure-map CC after a successful run is

cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb

- cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb
- For structure samples among those with top-ranked CCs, use

extracted_<x>_ps.pdb

- extracted_<x>_ps.pdb

Example

Limitation

Gromacs4.5.5 does not handle water molecules in structure files. Cryo_fit will remove any water from the input coordinate before starting.

Cryo_fit doesn't handle less common monomers such as 7C4, BMA, GDP, ILX, NAG, SEP, TRX. It will simply exclude those monomers and report what are excluded.

How to generate and record movie?

Generate_record_movie_by_cryo_fit1

Options

Default options will be used if unspecified. Expert users of Gromacs may want to customize some options if needed.

List of the most useful options

List of all available keywords

{{phil:cryo_fit.command_line.run}}

Tutorial: Fitting Biomolecules into Cryo-EM Maps using MD Simulation (GUI)

cryo_fit_gui.html

Contents

- Overview
- Input files
- Launch cryo_fit
- Enter Input Files
- Specify cryo_fit executable location
- Enter Options
- Run
- Result
- Output Example
- Limitation
- Reference
- How to generate and record movie?
- List of most useful options

Overview

This tutorial will show you how to fit biomolecular structures into cryo-EM maps with molecular dynamics (MD) simulation using Cryo_fit in the PHENIX graphical user interface (GUI).

For command-line tutorial, please see the [cryo_fit_commandline_tutorial](#)

Theoretical explanation of cryo_fit is [here](#)

For Cryo_fit installation, please see the [installation notes for cryo_fit](#)

Input files

<initial_model> and <target_map>

Initial Model: .cif or .pdb

The initial model is a guide or template structure (CIF/mmCIF/pdb) that is close to the model representing the target cryo EM map structurally.

Target Map: .ccp4, .map/.mrc (MRC style binary file), or .sit (Situs style text file)

Launch cryo_fit

Enter Input Files

Click browse buttons.

(Tutorial input files are in \$PHENIX/modules/cryo_fit/tutorial_input_files)

Specify cryo_fit executable location

(This step is optional if you've already setup your Cryo_fit environment in your .cshrc/.bashrc as instructed during Cryo_fit installation.)

Click browse buttons.

(select for example, \$PHENIX/modules/cryo_fit/gromacs/bin)

Enter Options

All options can be left blank (cryo_fit will figure out all options automatically).

Run

It should start something like this (total steps are 1~8).

Running time (With 2.7 GHz CPU, macOS)

All default values, it took ~2 minutes. With 10k steps, it took ~9 minutes

Result

Output files are in steps/8_cryo_fit folder. These include --

The final fitted atomic model: cryo_fitted.pdb

The .gro and .pdb files from the structures with the highest 3 map-model correlation coefficients (CC):
extracted_x_steps_x_ps.gro/pdb (.gro files are for visualization with vmd)

The history of CC values: steps/8_cryo_fit/cc_record

Output Example

Limitation

Gromacs4.5.5 does not handle water molecules in structure files. Cryo_fit will remove any water from the input coordinate before starting.

Cryo_fit doesn't handle less common monomers such as 7C4, BMA, GDP, ILX, NAG, SEP, TRX. It will simply exclude those monomers and report what are excluded.

Reference

S. Kirmizialtin, J. Loerke, E. Behrmann, C. MT. Spahn, K. Y Sanbonmatsu, Using Molecular Simulation to Model High-Resolution Cryo-EM Reconstructions, Methods Enzymol., 558, 2015, 497-514

How to generate and record movie?

Generate_record_movie_by_cryo_fit1

List of most useful options

Install Cryo_fit

cryo_fit_install.html

Contents

- [prerequisite] Install Phenix
- 1. Download Cryo_fit installer
- 2. Install Cryo_fit
- 4. Run Cryo_fit
- 6. Test run (optional)

[prerequisite] Install Phenix

Cryo_fit will be installed in your Phenix environment. Make sure you have Phenix installed and sourced. For Phenix installation, see installation notes for Phenix

1. Download Cryo_fit installer

Cryo_fit can be run on Linux or macOS computers. Please download Cryo_fit installer for Linux or Cryo_fit installer for macOS

2. Install Cryo_fit

Unzip the installer file, change to the extracted directory, and type `./install<enter>`. That's it. Please ensure that your account has 'write' privilege in your Phenix directory, or try `'sudo -E ./install'` if you have sudo access.

As a sequence of commands to execute (assuming the installer file is "Cryo_fit-Linux.zip"):

```
unzip Cryo_fit-Linux.zip
cd Cryo_fit-Linux
./install
```

At the end of installation, please follow the prompt to setup the path before starting Cryo_fit.

4. Run Cryo_fit

If you are in a Phenix session, please close and re-launch Phenix. You are now ready to run Cryo_fit in GUI or in command line.

6. Test run (optional)

You may want to know more about Cryo_fit. Try a test run! See the cryo_fit tutorial

How to generate and record movie by cryo_fit1?

cryo_fit_movie.html

Contents

- Pymol (confirmed with MacPymol 2.2.2)
- UCSF Chimera (confirmed with version 1.13 and 1.14)
- UCSF ChimeraX (confirmed with version 0.7 and 0.8 (2018-08-14))
- VMD (confirmed with version 1.9.3)
- Movie examples that are generated by cryo_fit
- Note

Pymol (confirmed with MacPymol 2.2.2)

- [command] cd output/trajectory
- [command] load trajectory.gro, mytraj
- [command] load_traj traj.xtc, mytraj
- Click salmon color play button (triangle) in right bottom to play

reference

UCSF Chimera (confirmed with version 1.13 and 1.14)

- [menu] Tools -> MD/Ensemble analysis -> MD Movie -> Trajectory format = GROMACS
- Run input (.tpr) = output/trajectory/for_cryo_fit.tpr
- Trajectory (.xtc) = output/trajectory/traj.xtc
- OK
- (optional)

[menu] Tools -> Depiction -> Rainbow -> OK [menu] File -> Open -> .../user.mrc (set opacity into 50%) [menu] Presets -> Publication 1

- [menu] Tools -> Depiction -> Rainbow -> OK
- [menu] File -> Open -> .../user.mrc (set opacity into 50%)
- [menu] Presets -> Publication 1
- (in MD Movie) click black play button to play
- (in MD Movie) File -> Record Movie, to record as .mp4 file

UCSF ChimeraX (confirmed with version 0.7 and 0.8 (2018-08-14))

- [menu] File -> Open -> .../output/cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobability.pdb
- [command] rainbow
- [menu] File -> Open -> .../user.map (set opacity into 50%)
- [command] cd <user_path>/output/trajectory

- [command] open traj.xtc structureModel #1

(note) Specified structure and coordinates should have the same number of atoms, if these are mismatched, Doo Nam recommends to use UCSF Chimera instead

- [command] coordset slider #1
- Click black play button to play
- Click red record button to record movie as .mp4 file

VMD (confirmed with version 1.9.3)

- [menu] File -> New Molecule -> Browse -> (.../user.map) -> Load
- [menu] Graphics -> Representation -> Draw -> Solid Surface
- [menu] Graphics -> Representation -> Material -> Transparent
- [menu] Graphics -> Representation -> Show -> Isosurface
- [menu] File -> New Molecule -> Browse -> (output/trajectory/trajectory.gro) -> Load

Movie examples that are generated by cryo_fit

- Watch supplementary videos in Journal of Structural Biology, Vol 208, 2019, p 1-6, Cryo_fit: Democratization of flexible fitting for cryo-EM

Note

- If the cryo_fit has been automatically repeated to improve cc, the trajectory file shows only the last cryo_fit run.
- Therefore, to watch a full change of conformations, "phenix.cryo_fit user.mrc user.pdb no_rerun=True number_of_steps_for_cryo_fit=<enough step>" is recommended.

Tutorial: Generating ligand coordinates and restraints for structure refinement

elbow.html

Introduction

This tutorial will demonstrate how to perform a structure refinement including a ligand.

Setup

You can create a subdirectory, 'elbow_tutorial', and copy the files required for this tutorial using the command:

```
elbow.setup_tutorial
```

When do we need restraints for a ligand?

A structure refinement requires that all atoms have energy types associated and ideally restraints to guide the geometry optimisation. PHENIX refinement uses a monomer library to resolve some ligands but not all.

Running on a structure containing an unknown ligand:

```
phenix.refine 1utn.pdb 1utn.mtz
```

results in the following error message.

```
Number of atoms with unknown nonbonded energy type symbols: 16
" N ABN A 246 "
" C ABN A 246 "
" C1 ABN A 246 "
" C2 ABN A 246 "
" C3 ABN A 246 "
" C4 ABN A 246 "
" C5 ABN A 246 "
" C6 ABN A 246 "
" N ABN A 247 "
" C ABN A 247 "
... (remaining 6 not shown)
```

Generating ligand restraints using PDB file

Clearly, we need to generate restraints for the ligand ABN which is in the PDB file. We commence by using the following command:

```
phenix.elbow 1utn.pdb
```

which prints out a list of the HETATM molecules in the PDB file.

```
Running eLBOW in expert mode
Defaults changed by /net/cci/nigel/.elbow/elbow_defaults.py
xyz : True (--no-xyz to reverse)
overwrite : True (--no-overwrite to reverse)
Random number seed: 578305348
0:00 Parsing Parsing Parsing Parsing Parsing Parsing Parsing Parsing P

Input format is PDB

MoleculeClass : C:11 N: 2 O: 3 (PDB format)
16 atoms
15 bonds
0 angles
```

```
0 dihedrals
```

```
Multiple residue names
resSeq resName #atoms type
246 'ABN' 8 HETATM
256 'TRS' 8 HETATM
```

Use --residue to specify residue

Use --do-all to process all residues not in internal library

As the print out says, all of the residues not in the internal library can be processed using

```
phenix.elbow 1utn.pdb --do-all
```

Because the residue TRS is in the internal library, only ABN is processed. The resulting files are elbow.ADB.1utn_pdb.cif and elbow.ABN.1utn_pdb.pdb. After checking the geometry in the PDB file, we repeat the calculation using the --opt to optimise the geometry using the semi-empirical AM1 method, thus:

```
phenix.elbow 1utn.pdb --do-all
```

The resulting file, elbow.1utn_pdb.all.cif, is ready for use with phenix.refine.

Generating ligand restraints using SMILES

To demonstrate the use of a SMILES string, we can get the SMILES string for ABN from the MSDChem website. We can now run using the SMILES string:

```
phenix.elbow --smiles "NCC1CCCCC1"
```

The resulting output, elbow.cif and elbow.pdb, have the default residue name, LIG, and atom names. In order to match the atoms in the refinement, we can use the PDB to obtain the atom names via the --template option and the --residue option:

```
phenix.elbow --smiles "NCC1CCCCC1" --template 1utn.pdb --residue ABN --opt
```

The resulting output files have the AM1 optimised geometry (--opt), the atom names from the PDB file and the correct connectivity from the SMILES string.

Generating link edits

The input PDB file, 1utn.pdb, has a number of LINK records. These can be converted to "edits" for input into phenix.refine. The LINK records:

```
LINK CA CA A6000 OE1 GLU A 70 1555 1555
LINK CA CA A6000 O ASN A 72 1555 1555
LINK CA CA A6000 O VAL A 75 1555 1555
LINK CA CA A6000 O HOH Z 89 1555 1555
LINK CA CA A6000 O HOH Z 104 1555 1555
LINK CA CA A6000 OE2 GLU A 80 1555 1555
```

Running:

```
phenix.link_edits 1utn.pdb
```

results in the "edits" being written to 1utn_pdb.edits. The "edits" are a one-to-one correspondence of the LINK records and a bond "edit".

Metal coordination

Conveniently, the LINK records were contained in the PDB file used in this example. More likely, the linking information is not available. The "edits" can be obtained using:

```
phenix.metal_coordination 1utn.pdb
```

The "edits" are written to elbow.edits and contain both bond and angle "edits".

Putting it all together

We can put all the files together to perform a refinement. Adding the option --dry-run reads the data. Remove to perform complete refinement.

```
phenix.refine 1utn.mtz 1utn.pdb elbow.1utn_pdb.all.cif 1utn_pdb.edits --dry-run
```